# SCALING MERGE SORT FROM SERIAL BASELINES TO BANDWIDTH-LIMITED PARALLELISM

## PROJECT PROPOSAL


NAME : K.G.N. RUVINDA

REGISTRATION NO : EN113603

INDEX NO : 23/ENG/122


SUBMISSION DATE : 21/10/2026

# TABLE OF CONTENTS

**1 Introduction** **1**

**2 Background** **2**

**3 Aims and Objectives** **3**

3.1 Aim . . . . . 3

3.2 Objectives . . . . . 3

**4 Implementation Plan** **4**

**5 Milestones and Timeline** **6**

**6 System Architecture Diagram** **9**

**7 References** **10**

# 1 INTRODUCTION

Sorting massive datasets is a fundamental computational primitive across database query optimizers, search backends, spatial indexing engines, and scientific simulations. While classical algorithm analysis models Merge Sort as having an optimal deterministic time complexity of $\mathcal{O}(n \log n)$, practical execution on modern multi-core microprocessors is dictated by physical memory architecture and cache subsystem constraints rather than theoretical arithmetic bounds alone.

On contemporary superscalar processors, arithmetic execution throughput scales with core count, but the off-chip memory subsystem (shared memory bus, Last-Level Cache, and DRAM channels) does not scale at a proportional rate. Because Merge Sort possesses a low arithmetic intensity, executing only a few comparison operations per cache line retrieved from main memory, it quickly encounters the "Memory Wall". Once memory bandwidth is fully saturated, spawning additional execution threads delivers diminishing performance returns, resulting in an unyielding scaling ceiling.

High-Performance Computing (HPC) requires understanding when and why software hits architectural boundaries. While standard parallel divide-and-conquer formulations scale effectively during the recursive divide phase, the subsequent merge operations remain an execution bottleneck. Without parallelizing the merge phase itself, Amdahl's Law strictly caps system speedup. Even when the merge phase is parallelized via concurrent multi-threaded partitioning, the algorithm shifts from being compute-bound to bandwidth-constrained.

This project proposes the design, implementation, and systems-level profiling of a parallel merge sort framework in modern C++. Rather than treating hardware as an abstract black box, this work evaluates the full spectrum of sorting execution: from verified serial baselines to dynamic task-based parallel divide implementations, culminating in concurrent binary-search partitioned merging. The system is benchmarked against production libraries and evaluated using hardware performance counters and empirical Roofline models to diagnose bandwidth-limited plateaus.

# 2 BACKGROUND

Merge sort operates via a recursive divide-and-conquer paradigm: recursively subdividing an unsorted array into halves until trivial leaf partitions are reached, and subsequently merging sorted segments back into a cohesive, ordered array.

Parallelizing this algorithm exposes critical microarchitectural trade-offs. Standard multi-threaded implementations restrict concurrency to the divide phase by spawning independent tasks for recursive branches. However, the recombining merge step remains predominantly sequential. As core counts expand, this sequential merge phase dominates total execution time, validating Amdahl's Law and severely restricting multicore parallel efficiency.

To break past this barrier, concurrent partitioned merging must be introduced. By employing dual binary-search rank partitioning, two sorted sub-arrays can be mathematically bisected into disjoint, non-overlapping segments. This allows multiple threads to execute the merge step concurrently without write collisions or lock contention.

However, solving the algorithmic synchronization challenge exposes the underlying hardware constraint: memory bandwidth exhaustion. Merging requires continuously streaming data out of two disjoint input arrays and streaming write traffic into an auxiliary scratchpad. This operation quickly exceeds the bandwidth capacity of shared Last-Level Caches (LLC) and DRAM channels.

A thorough systems investigation requires measuring the point at which an algorithm transitions from being scheduling-bound to memory-bound. Profiling cache line eviction rates, thread migration penalties, and memory bus saturation through Performance Monitoring Units (PMUs) provides the empirical evidence necessary to explain why scaling curves flatten on high-core-count architectures.

# 3 AIMS AND OBJECTIVES

## 3.1 Aim

To design, implement, and rigorously analyze a high-performance parallel merge sort framework in modern C++, evaluating its scaling behavior against microarchitectural ceilings and memory bandwidth saturation on multi-core systems.

## 3.2 Objectives

1. **Baseline Development:** Implement an optimized, sequential reference merge sort in modern C++ with an empirical leaf cutoff to `std::sort` and a pre-allocated auxiliary scratchpad buffer.

2. **Correctness & Verification Harness:** Build an automated testing framework to validate sort correctness, sequence invariants, and numerical stability across uniform random, pre-sorted, reverse-sorted, organ-pipe, and duplicate-dense test distributions.

3. **Task-Parallel Divide Implementation:** Parallelize the recursive division phase using OpenMP tasking (`#pragma omp task`) configured with a single root task region to minimize thread team allocation overhead.

4. **Amdahl Bottleneck Evaluation:** Quantify the empirical serial fraction and performance plateau of the parallel-divide/serial-merge implementation across scaling thread counts $(1, 2, 4, 8, 16)$.

5. **Concurrent Binary-Search Partitioned Merge:** Derive and implement a parallel merge kernel that utilizes dual binary-search rank partitioning to facilitate concurrent writes into non-overlapping memory slices without data races.

6. **Comparative Library Benchmarking:** Evaluate custom implementations against industry standard baselines, including `std::sort` and Intel OneAPI TBB `tbb::parallel_sort`, across sizes from $n = 10^3$ to $10^8$ elements.

7. **Hardware Counter Profiling:** Instrument the execution pipeline using Linux `perf stat` to record hardware event counters, including Last-Level Cache (LLC) misses, Instructions Per Cycle (IPC), and cache bus contention.

8. **Roofline Systems Modeling:** Construct an empirical Roofline Model chart to quantitatively evaluate the sorting kernel's operational intensity against the system's peak DRAM bandwidth ceiling.

# 4 IMPLEMENTATION PLAN

The framework will be developed as a standalone, modular C++20 systems application running on an Ubuntu Linux environment, built via CMake and strictly decoupled into algorithmic engines, profiling harnesses, and data visualizers.

**Technology Stack:**

- **Core Language & Concurrency:** Modern C++ (C++20) utilizing OpenMP 5.0+ tasking paradigms and thread affinity controls (`OMP_PROC_BIND=spread,close`).
- **Compiler Infrastructure:** GCC / Clang configured with maximum hardware optimizations (`-O3`, `-march=native`, `-flto`, `-fopenmp`) and strict warning compliance.
- **Baseline Comparative Suite:** Intel OneAPI Threading Building Blocks (`oneTBB`) for industry-standard parallel work-stealing baselines.
- **Benchmarking & Systems Profiling:** Linux `perf` subsystem (`perf stat`, `perf record`), Likwid PMU tools, and monotonic timers (`std::chrono::steady_clock`).
- **Telemetry & Analytics:** Python 3 (NumPy, Pandas, Matplotlib, Seaborn) for processing telemetry CSV files and plotting speedup, Amdahl curves, and Roofline charts.

**Architectural Flow:**

1. **Distribution Generation:** Synthesizes test vectors across sizes $n = 10^3$ through $10^8$ with varying entropy profiles (random, organ-pipe, sorted, reverse, duplicates).
2. **Harness Dispatcher:** Prepares auxiliary memory buffers, warms processor instruction/data caches, and dispatches arrays to the selected sorting engine.
3. **Algorithmic Engine Execution:**
   - *Variant 1 (Serial Baseline):* Recursively bisects data until leaf cutoff ($k \approx 32$–$64$), invoking `std::sort` and merging serially.
   - *Variant 2 (Parallel Divide):* Spawns concurrent OpenMP tasks during recursive descent, merges partitions serially per node.
   - *Variant 3 (Parallel Merge):* Employs dual binary search rank partitioning to identify disjoint segments, recursively executing both divide and merge phases concurrently.
4. **Correctness Verification:** Confirms monotonic ordering ($\forall i : A[i] \leq A[i+1]$), validates multiset checksum parity against `std::sort`, and checks key-value payload stability.

5. **Hardware Performance Profiling:** Executes wrapped runs under `perf stat` to sample LLC misses, instructions retired, and memory bus access volumes.

6. **Telemetry & Synthesis:** Outputs benchmark telemetry to CSV files for statistical analysis (computing medians and 95% confidence intervals) and generates empirical Roofline performance models.

# 5 MILESTONES AND TIMELINE

The project will be executed across two semesters. Semester 6 prioritizes algorithmic foundations, correctness frameworks, and baseline parallelization, while Semester 7 focuses on concurrent partitioned merging, hardware counter profiling, and roofline analysis.

## Semester 6: Algorithm Development and Baseline Integration

| Period | Planned Activities | Milestone |
|---|---|---|
| Weeks 1–2 | Configure Ubuntu development environment, CMake build system, Git repo, and compiler optimization flags. | Environment verified. |
| Weeks 3–4 | Implement input distribution generators (uniform, sorted, reversed, organ-pipe, duplicates) and timing utilities. | Test vector suite ready. |
| Weeks 5–6 | Implement serial recursive merge sort; calibrate empirical leaf cutoff to `std::sort`; build scratchpad manager. | Variant 1 serial baseline. |
| Weeks 7–8 | Build validation harness asserting sortedness invariants, checksum parity, and key-payload stability. | Verification harness. |
| Weeks 9–10 | Implement OpenMP tasking recursive divide; configure single root task parallel region to limit runtime overhead. | Variant 2 parallel divide. |
| Weeks 11–12 | Implement task depth cutoffs; benchmark Variant 2 across core counts ($1 \rightarrow 16$); record initial speedups. | Divide scaling profiled. |
| Weeks 13–14 | Quantify empirical serial fraction; formulate Amdahl's Law model for serial merge step; submit interim report. | Semester 6 review. |

## Semester 7: Advanced Concurrency, Hardware Profiling, and Synthesis

| Period | Planned Activities | Milestone |
|---|---|---|
| Weeks 1–2 | Derive mathematical invariants for dual binary-search partitioning; implement rank-splitting algorithms. | Partitioning logic derived. |
| Weeks 3–4 | Implement recursive concurrent merge tasks writing into disjoint memory slices; verify data-race freedom. | Variant 3 engine complete. |
| Weeks 5–6 | Integrate Intel OneAPI TBB `parallel_sort` baseline; execute scaling sweeps across sizes $10^3 \rightarrow 10^8$. | Comparative suite tested. |
| Weeks 7–8 | Run multi-trial benchmark suites; compute medians, variance, and 95% confidence intervals to isolate OS jitter. | Scaled benchmark data. |
| Weeks 9–10 | Instrument Linux `perf stat` / Likwid; record LLC cache misses, instructions per cycle, and memory traffic. | Microarchitecture telemetry. |
| Weeks 11–12 | Construct empirical Roofline Model; correlate memory bus saturation with speedup plateaus across thread counts. | Bandwidth analysis complete. |
| Weeks 13–14 | Finalize telemetry visualizations, author complete dissertation report, and prepare project demonstration deck. | Final submission. |

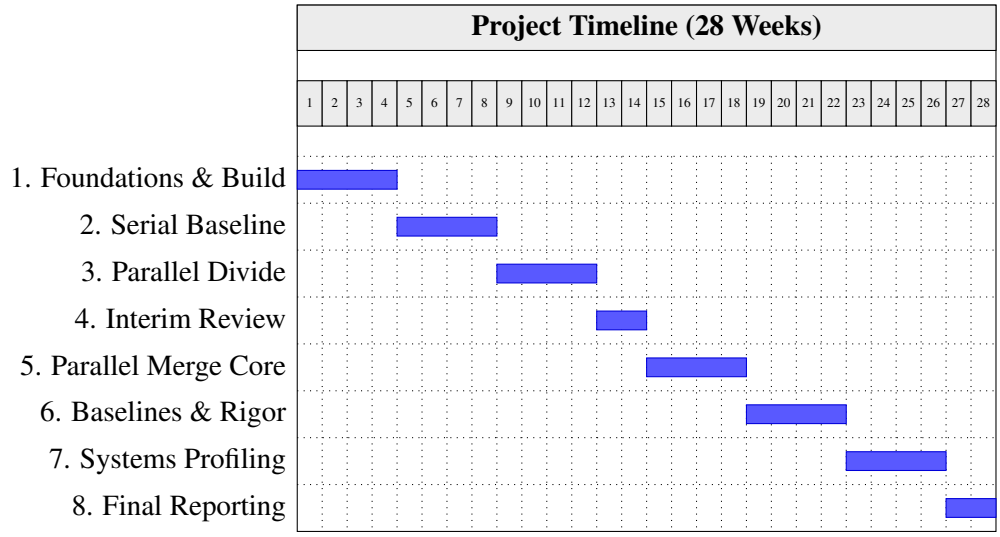



Figure 1: Project timeline Gantt Chart

# 6 SYSTEM ARCHITECTURE DIAGRAM

The following diagram illustrates the structural pipeline, execution hierarchy, data-flow routing, and hardware profiling pathways across the three sorting variants.

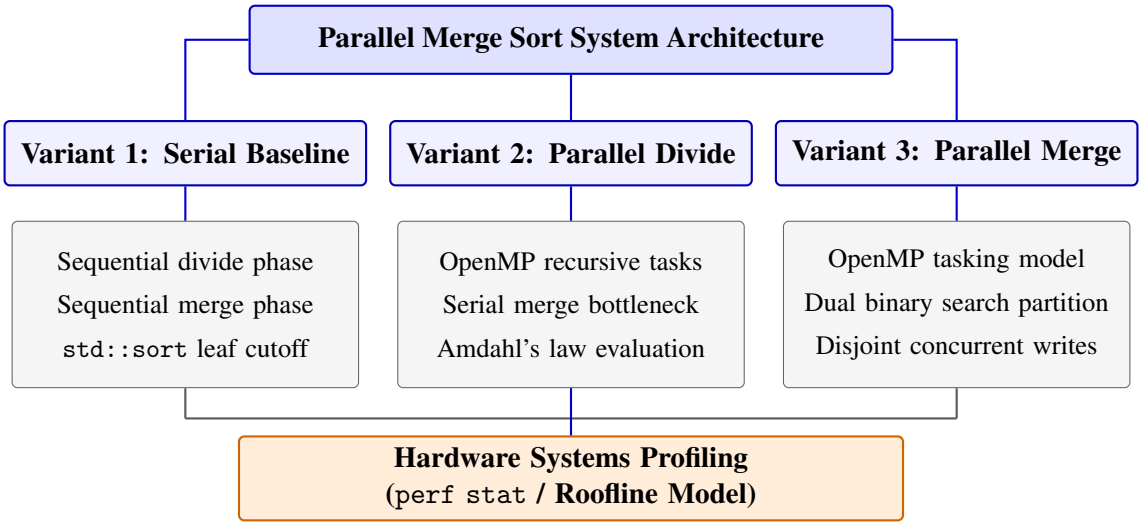



Figure 2: System architecture and progression across the three sorting implementations.

# 7 REFERENCES


1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

2. S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

3. OpenMP Architecture Review Board, "OpenMP Application Programming Interface Specification Version 5.2," *OpenMP.org*, Nov. 2021.

4. Intel Corporation, "Intel OneAPI Threading Building Blocks (oneTBB) Documentation," *Intel Software*, 2024. [Online]. Available: https://uxlfoundation.github.io/oneTBB/.

5. P. Sanders, K. Mehlhorn, M. Dietzfelbinger, and R. Dementiev, *Sequential and Parallel Algorithms and Data Structures: The Basic Toolbox*. Cham, Switzerland: Springer, 2019.

6. U. Meyer, P. Sanders, and J. Sibeyn, Eds., *Algorithms for Memory Hierarchies: Advanced Lectures*, LNCS 2625. Berlin, Germany: Springer-Verlag, 2003.

7. V. Weaver, "Linux perf Examples and Hardware Performance Event Counters," *University of Maine Technical Report*, 2023.

8. G. E. Blelloch, "Programming Parallel Algorithms: Merge Sort and Sorting Networks," *Communications of the ACM*, vol. 39, no. 3, pp. 85–97, 1996.